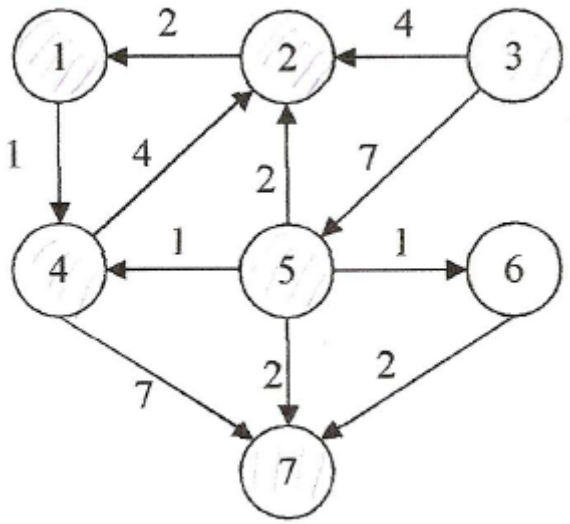
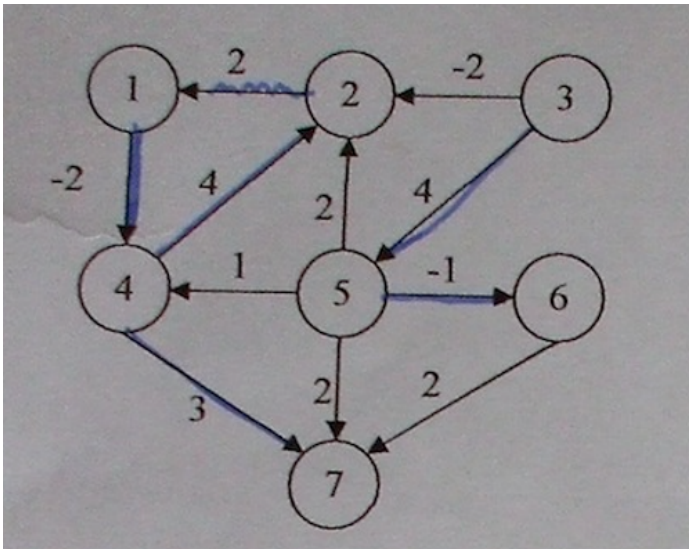
Subiectul 1:

1. Se da graful din figura alaturata
   a. Sa se aplice o parcurgere in latime pornind din nodul 3 (ignorand costurile asociate muchiilor).
   b. Sa se aplice algoritmul Dijsktra pornind din nodul 3, precizand pentru fiecare pas nodurile din coada de prioritati (si valorile asociate acestora) si nodul ales pentru a fi scos din aceasta.
   c. Sa se aplice algoritmul A*, considerand starea initiala nodul s = 3 si starea finala nodul t = 7. euristica folosita este h(v) = {max (cost(v, v')) | (v, v') muchie din graf} pentru v != t si h(t) = 0. sa se calculeze h si sa se evidentieze la fiecare pas multimile OPEN, CLOSED, precum si modificarile asupra lui f si g. Va puteti opri dupa primii 4 pasi.



   d. Aplicati primii 3 pasi ai algoritmului Floyd-Warshall, evidentiind valorile matricelor D(0), D(1), D(2), D(3). La fiecare pas este suficient sa spuneti care elemente din matricea de la pasul urmator de va modifica.

2. Se da graful din figura alaturata
   a. Sa se aplice o parcurgere in adancime evidentiind pentru fiecare nod timpul de descoperire si de finalizare.
   b. Pentru parcurgerea anterioara precizati tipul fiecarei muchii din graf
   c. Pornind de la parcurgerea de la punctul a, determinati componentele tare conexe ale grafului, parcurgand graful transpus si calculand timpii de descoperire si de finalizare
   d. Aplicati primii 3 pasi ai algoritmului Floyd-Warshall, evidentiind valorile matricelor D(0), D(1), D(2), D(3). La fiecare pas este suficient sa spuneti care elemente din matricea de la pasul urmator de va modifica.



   e. Pozitivati costul muchiilor negative prin considerarea modulului acolo unde este cazul, determinati fluxul maxim in graf folosind algoritmul Edmonds-Karp, stiind ca sursa este nodul 3, iar drena nodul 7. La fiecare pas, specificati varfurile ce fac parte din drumul de ameliorare ales de algoritm, precum si cantitatea de flux care poate fi transportata pe acest drum.
   f. considerati graful neorientat si pozitivand costul muchiilor negative prin considerarea modulului acolo unde este cazul, aplicati algoritmul lui Prim pornind din nodul 5 (la fiecare pas, evidentiind valoarea fiecarui nod aflat in coada de prioritati, precum si nodul extras din coada).
   g. Aplicand primii pasi ai algoritmului lui Johnson, calculati potentialul fiecarui nod si recalculati costurile muchiilor astfel incat sa le pozitivati conform formulei folosite de algoritm

3. Se da graful din figura alaturata
    a. Sa se determine o sortare topologica a grafului.
    b. Pozitivati costul muchiilor negative prin considerarea modulului acolo unde este cazul si aplicati algoritmul A*, considerand starea initiala a nodului s = 1 si starile finale nodurile t1 = 6 si t2 = 7. Euristica folosita este h(v) = {max (cost(v, v')) | (v, v') muchie din graf} pentru v != t1 si t2 si h(t1) = h(t2) = 0.
    c. Considerand graful neorientat sa se identifice punctele de articulatie din graf.
    d. pozitivand costul muchiilor prin modul, aplicati algoritmul Kruskal

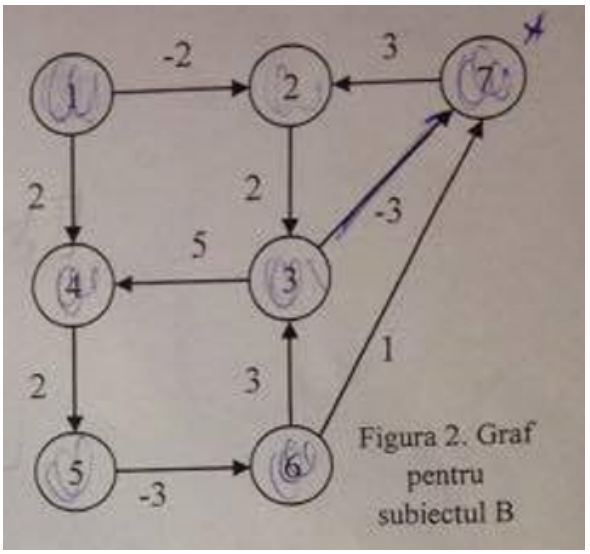


Figura 2. Graf pentru subiectul B

Subiectul 2:

1. **Comparati tehnicile greedy si programare dinamica, evidentiind asemanarile si deosebirile, scotand in evidenta cand aceste tehnici ofera solutii optime pentru problema de rezolvat.**
   greedy
   - bottom-up - pleaca de la un optim local catre catre un optim global
   - se tine cont doar de pasul anterior
   - problema trebuie sa aiba substructura optima (solutia problemei trebuie sa contina solutiile subproblemelor
   - problema trebuie sa aiba caracter de matroid
   - implica de multe ori o sortare
   - solutiile partiale ce nu pot fi imbunatatite sunt abandonate.
   -

   programare dinamica
   - bottom-up - Soluţii optime construite iterativ asamblând soluţii optime ale unor subprobleme (probleme similare de dimensiune mai mică)
   - se tine cont de toti pasii anteriori (memorizare)
   - are nevoie de cele mai multe ori de o structura suplimentara in care sa se retina solutiile anterioare

2. **Descrieti ce imbunatatiri aduce tehnica taieturilor alpha-beta algoritmului minimax, explicand implicatiile asupra complexitatii rezolvarii. Dati un exemplu de arbore binar de joc de inaltime cel putin 2 care sa evidentieze cel putin o taietura rezultata prin aplicarea alfa-beta.**

3. **Care este algoritmul generic pentru determinarea arborelui minim de acoperire al unui graf? Cum pot fi derivati algoritmii Prim si Kruskal pornind de la acest algoritm generic?**
   Algoritmul general pentru determinarea AMA se bazeaza pe identificarea unei muchii sigure si adaugarea ei in AMA partial. O muchie sigura este una care adaugata la un AMA partial atunci multimea de muchii alese ramane un AMA.

```
T = { }
while T nu este un AMA
        aleg o muchie sigura
        T = T U {muchie}
for (u, v) muchie
        if (u, v) = muchie sigura si nu este in AMA
                adauga (u, v) la AMA
```

   Prim:

- porneste de la un nod si extinde pe rand cu muchiile cu cost minim care au un singur capat in AMA deja format
- asemanator lui Dijkstra

Kruskal:

- initial toate nodurile dormeaza cate o multime si la fiecare pas se reunesc 2 multimi printr-o muchie
- muchiile sunt considerate in ordinea costurilor si adaugate in AMA daca nu formeaza ciclu.

4. **Explicati utilitatea si functionarea algoritmului Johnson. de ce este mai bun acest algoritm decat alternativele?**

   Johnson:

   - este pentru gasirea drumurilor de cost minim multipunct-multipunct
   - este mai bun pentru grafuri rare (fata de Floyd-Warshal)
   - functioneaza pe grafuri cu muchii cu cost negativ
   - detecteaza cicluri de cost negativ (in etapa de Bellman-Ford)
   - complexitate :
   - functionare:
     - se adauga un nod fictiv care se leaga de toate celelelte noduri prin muchii de cost 0
     - se aplica Bellman-Ford pentru calcularea potentialelor fiecarui nod (h(v) = d(v) returnat de Bellman-ford plecand din nodul fictiv
     - se elimina nodul fictiv si muchiile adiacente lui
     - se recalculeaza consturile fiecarei muchii w'(u, v) = w(u, v) + h(u) - h(v)
     - se aplica Dijkstra din fiecare nod in fiecare nod
     - se recalculeaza distantele adevarate d'(v) = d(v) + h(v) - h(u) (unde d(v) este distanta calculata pana la v din sursa u cu Dijkstra)

5. **Care este complexitatea algoritmului A*? Cum este aceasta complexitate fata de complexitatea algoritmilor neinformati? Explicati.**

   - liniara daca dacă $|h^*(n) - h(n)| \leq \delta$, unde $\delta \geq 0$ este o constantă unde h^* costul exact al porțiunii de drum optim n.. γ , pentru cel mai favorabil nod, iar costul estimat al drumului optim de la nodul n la cel mai favorabil nod soluție γ
   - Subexponențială , dacă $|h^*(n) - h(n)| \leq O(\log(h^*(n)))$.
   - Exponențială , altfel, (dar mult mai bună decât a căutărilor neinformate)

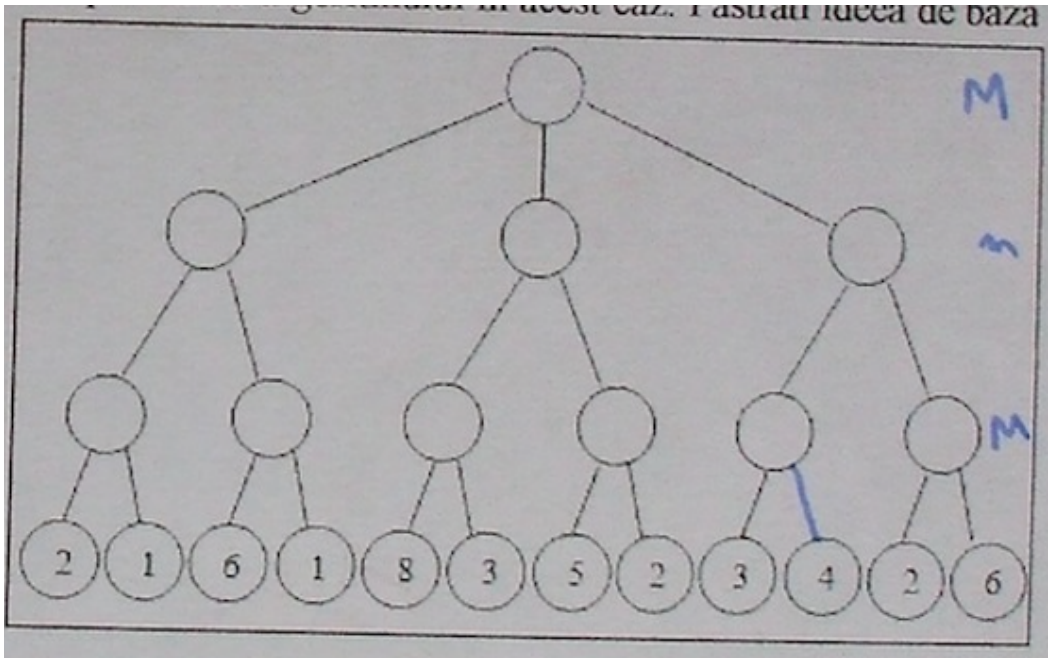
6. **Comparati algoritmii AC1 si AC3, apoi propuneti o modalitate simpla de a imbunatati si mai mult performanta (temporala) a acestui tip de algoritmi.**

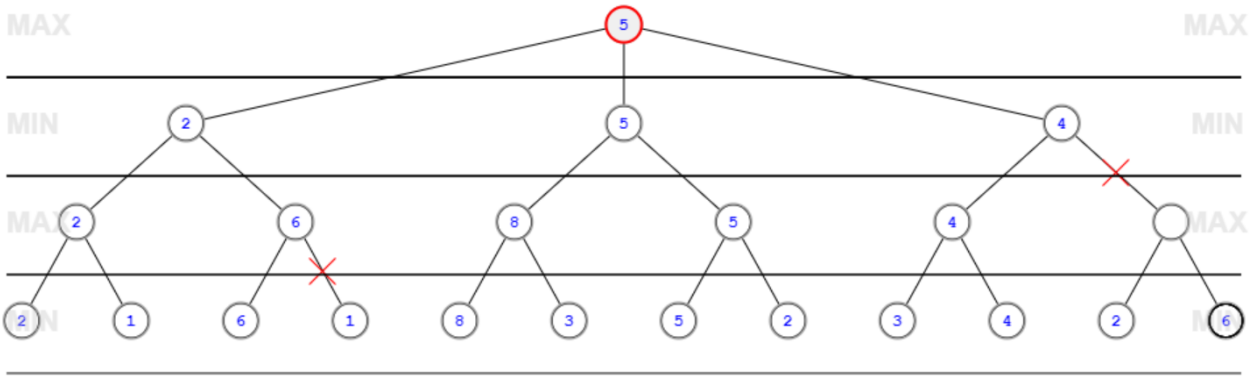
7. Cum poate fi adaptat algoritmul lui Dijsktra pentru a functiona pentru grafuri orientate care contin muchii de cost negativ, dar nu contin cicluri de cost negativ? Spuneti cum se modifica algoritmului in acest caz. Pastrati ideea de baza a algoritmului Dijkstra (alegerea greedy)!
8. Ce este o euristica consistenta? Ce este o euristica admisibila? Demonstrati ca o euristica consistenta este admisibila
9. **Care sunt pasii pe care trebuie sa ii urmati pentru a rezolva o problema prin programare dinamica? Exemplificati acesti pasi pentru problema parantezarii matricelor studiata la curs, spunand intai de toate cerinta acestei probleme**

   Aplicarea acestei tehnici de programare poate fi descompusă în următoarea secvență de pași:

   10. Identificarea structurii și a metricilor utilizate în caracterizarea soluției optime;
   11. Determinarea unei metode de calcul recursiv pentru a afla valoarea fiecărei subprobleme;
   12. Calcularea "bottom-up" a acestei valori (de la subproblemele cele mai mici la cele mai mari);
   13. Reconstrucția soluției optime pornind de la rezultatele obținute anterior.

14. care este numarul minim, respectiv maxim de relaxari efectuate de catre algoritmul Bellman-Ford pentru fiecare muchie a unui graf orientat? Dar pentru Dijsktra?
15. Pornind de la definitie, explicati de ce este important ca euristica folosita de A* sa fie admisibila? Demonstrati mai apoi ca orice euristica consistenta este si admisibila.
16. Definiti ce inseamna graful rezidual al unei retele de transport si demonstrati (chiar si informl) ca atunci cand nu mai exista nici o cale de ameliorare de la sursa la drena inseamna ca am descoperit taietura minimala a retelei de transport.
17. Sa se aplice algoritmul minimax cu imbunatatirea alpha-beta pentru arborele de joc din figura alaturata, specificand atat valorile pentru fiecare nod, cat si ramurile taiate de catre alpha-beta.





18. Care sunt asemanarile si diferentele intre algoritmii de tip Las Vegas si de tip Monte Carlo?
19. Cum se detecteaza ciclurile negative la Bellman-Ford?
20. Care sunt diferentele intre backtracking si programare dinamica?
21. Transformand unul din algoritmii facuti la curs, scrieti in pseudocod un algoritm de detectare eficienta a inchiderii tranzitive a unui graf neorientat. Justificati. Ce complexitate are?
22. Ce este o cale reziduala si cum se poate obtine? Care este capacitatea reziuduala a caii si cum se poate determina. Exemplificati.
23. Ce este o euristica consistenta? Ce este o euristica monotona? Demonstrati ca o euristica este consistenta daca si numai daca este monotona.
24. Dati exemplu de 3 algoritmi Greedy facuti la curs
    R: Prim, Kruskal si Dijsktra
25. Comparati BF* cu BFS.
26. Demonstrati corectitudinea algoritmului Kruskal
27. Demonstrati corectitudinea algoritmului Prim

UITE AICEA:

SE VEDE?????

28. Comparati tehnicile de programare Divide et impera si Greedy

Divide et impera:

- de cele mai multe ori se rezolva folosind recursivitatea si ocupa mai mult spatiu pe stiva
- top-down:
- se imparte problema mare in probleme mai mici care se imparte in probleme mai mici care sunt rezolvate recursiv, iar apoi solutiile sunt combinte pentru a obtine rezultatul final
- rezolvam toate subproblemele pe care le generam (poate rezolva aceeasi problema de mai multe ori)
- nu are un criteriu de optim

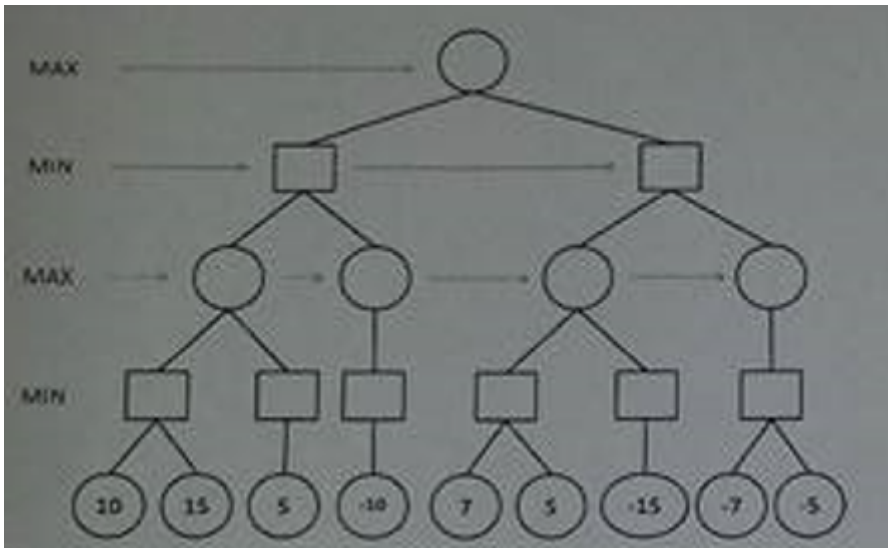
Greedy:

-

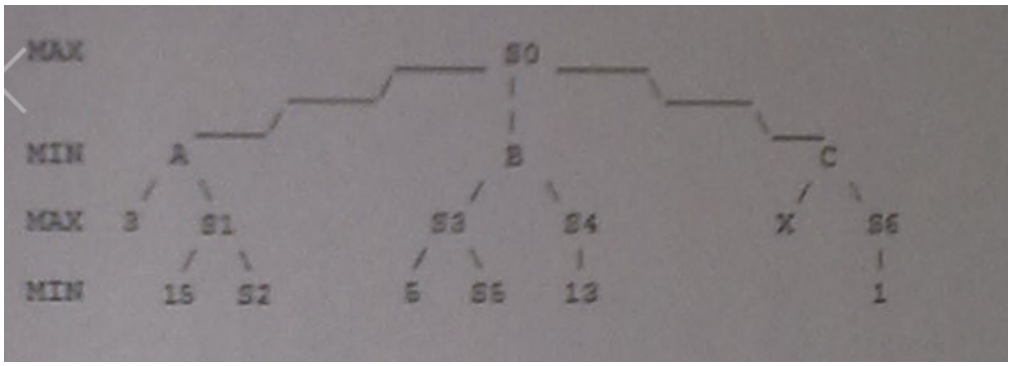
R: Ambele tehnici au ca idee de rezolvare plecarea de la o problema mare si impartirea ei in subprobleme. La divide et impera este clara aceasta impartire, iar la Greedy adaugand o solutie partiala la solutia finala ramane o problema mai mica de rezolvat. Astfel, ambele tehnici de programare au o abordare top-down pentru ca pleaca de la o problema mai mare si rezolva mai apoi prrobleme mai mici.

La divide et impera nu avem un criteriu de optim, rezolvam toate subproblemele pe care le generam, in timp ce la Greedy rezolvam doar o problema, cea care returneaza un optim local, iar solutiile partiale ce nu pot fi imbunatatite sunt abandonate.

29. Care sunt asemanarile si diferentele intre algoritmii de tip BF* si A*?
30. Calculati valoarea MINIMAX a arborelui de joc urmator



31. Care sunt modalitatile de reprezentare a unei PSR (problema de satisfacere a restrictiilor)?
32. Care este valoarea maxima a lui X din arborele urmator astfel incat subarborii cu radacina in A si C sa fie taiati prin alpha-beta?



33. Definiti ce inseamna graful rezidual al unei retele de transport si demonstrati (chiar si informal) ca atunci cand nu mai exista nicio cale de ameliorare de la sursa la drena in graful rezidual, inseamna ca am descoperit taietura minimala a retelei de transport
34. Dati exemplu de o problema care se poate rezolva, eventual cu constrangeri diferite asupra datelor de intrare, atat folosind programare dinamic, cat si greedy. Explicati diferentele si asemanarile intre cei 2 algoritmi (inclusiv complexitate)
35. Aplicati minimax si evidentiati taieturile alfa-beta pentru arborele de joc alaturat. Influenteaza ordinea de evaluare a nodurilor numarul de taieturi dintr-un arbore de joc?
36. Care este numarul minim, respectiv maxim de componente conexe ale unui graf care admite o sortare topologica? Justificati
37. Eplicati pe scurt principiul de functionare al algoritmului de pompare preflux. care este inaltimea maxima la care poate ajunge un nod in timpul rularii algoritmului? De ce este utila/ necesara aceasta limita?
38. Dandu-se matricea finala de distante obtinuta folosind Floyd-Warshall se poate stabili daca graful resectiv are un ciclu de cost negativ? Argumentati.
39. Demonstrati ca un graf are cicluri daca si numai daca in timpul executiei DFS sunt descoperite arce inapoi.
40. **Evidentiati etapele cele mai importante ale stategiei de rezolvare divide et impera, dupa care explicati cum poate fi folosita aceasta pentru a calcula eficient x pow n (x^n) pentru x real, iar n >= 0**

    R: Divide et impera reprezinta o clasa de probleme care au ca principale caracteristici faptul ca impart problema in subprobleme similare cu problema initiala, dar mai mici ca dimensiune, rezolva problemele recursiv si apoi combina solutiile pentru a rezolva problema originala

    Pasi:
    - imparte problema in mai multe subprobleme
    - subproblemele sunt rezolvate recursiv (daca sunt destul de mici se rezolva imediat)
    - solutiile sunt combinate pentru obtinerea rezultatului final

    calcularea x^n:
    - daca n este par problema se imparte in calcularea recursiva a lui x^(n/2) si la final se face calculul x^(n/2) * x^(n/2)
    - daca n este impar se imparte la fel problema, iar rezultatul final se obtine facand x * x^(n/2) * x^(n/2)
    - complexitate = O(log n)

41. Explicati utilitatea algoritmilor de satisfacere/propagare a constrangerilor, pornind de la functionalitatea oferita de REVISE. Trecand apoi in revista algoritmul AC1, explicati de ce acesta este considerat ineficient
42. Definiti notiunea de dominanta pentru A*, explicand de ce aceasta este importanta in alegerea euristicii. Dati exemplu de o problema unde puteti defini (cel putin) 2 euristici diferite si discutati care dintre ele este dominanta
43. Dati un exemplu de un algoritm studiat la curs care poate fi utilizat pentru determinarea ciclurilor de cost negativ intr-un graf orientat. Dupa aceea, scrieti pseudocodul pentru determinarea tuturor nodurilor care alcatuiesc un astfel de ciclu SAU demonstrati corectitudinea determinarii ciclului negativ de catre acest algoritm

44. Algoritmul DFS poate fi folosit pentru clasificarea muchiilor unui graf orientat, Specificati tipurile de muchii si cum se poate determina tipul fiecarei muchii din graf folosind o singura parcurgere in adancime.

45. Ce este un matroid?

R: Un sistem accesibil este un matroid daca satisface proprietatea de interschimbare: oricare X si Y apartin I si |X| < |T| => exista e care apartine Y / X astfel incat X U {e} apartine I

DIVIDE ET IMPERA		**GREEDY**		**PROGRAMARE DINAMICA**		**BACKTRACKING**			**AMA - Arbori de Minima Acoperira**		
TOP-DOWN		**TOP-DOWN**		**BOTTOM-UP**							
Merge Sort	O(NlogN)	Arbori Huffman		AOC	O(n^3)	PSR (Probleme de satisfacere a restrictiilor)	NC (Node Consistency)	NC(i) - pentru un singur i	Prim	Matrice de adiacenta	O(n^2)
Calculul puterii unui numar	O(logN)	Problema Rucsacului	Merge doar la greutati care se pot fractiona		O(n^2) - Cu optimizare KNUTH			NC-1 - pentru fiecare I de la 1 la n			Pentru grafuri dese
Caclulul celei mai scurte distante intre doua puncte din plan	O(NlogN)	Explorare Lacoma		Numere Catalane				O(na)		Heap Binar	O(ElogV)
		Dijkstra	O(V^2) - Brut	Chain Matrix Multiplication (Parantezare optima de matrici)	O(n^2) - Spatiu		Revise	Exemplu de NC-1		Heap Fibonacci	O(VlogV + E)
			O(ElogV) - cu Heap Binar		O(n^3) - Timp			O(a^2)	Kruskal	O(ElogV)	
			O(VlogV + E)	Floyd-Warshall	O(V^3) - Timp		AC-1 (Arc Consistency)	Foloseste NC-1	Flux	Pompare Preflux	O(V^2*E)
			Merge cand nu avem muchii negative		O(V^3) - Spatiu			Foloseste Revise		Arc rezidual	f(u,v) < cost(u, v)
		Bellman Ford	O(VE)		O(V^2) - Spatiu optimizat cu Inchidere Tranzitiva			O(na * 2r * a^2)		Ford-Fulkerson	O(E*fmax)
			Detectie Cicluri	Viterbi			AC-3	Mai putine apeluri Revise		Edmonds-Karp	Imbunatatire a Ford-Fulkerson
								O(a^3*r)			Contine BFS
											O(E^2*V)

GRAFURI						**ALTE MIZERII**		**MINIMAX**
Algoritmi			**Notiuni**			**ALGORITMI ALEATORI**		
BFS	FIFO		Punct de articulatie	Exista x, y a.i. orice drum de la x la y trece prin nodul u (punctul de articulatie)		Las Vegas	Gaseste solutia optima	Arbore AND-OR
	O(N+M)						O(g(n))	Talere alpha-beta
	Nu parcurge tot graful		Punte	Exista x, y a.i. orice drum de la x la y trece prin muchia plm (care e punte)		Monte Carlo	Solutie Incorecta	Solutie Posibilia
DFS	LIFO						O(g(n))	
	O(N+M)					**Explorare informata irevocabila**		
	Parcurge tot graful					Gradient Maxim	ex: Simple Trace O(bd)	
Dijkstra	O(V^2) - Brut	Grafuri Rare				**Explorari tentative informate**		
	O(ElogV) - cu Heap Binar	Grafuri Dese				Explorare Lacoma	Nu e complet	
	O(VlogV + E)					BF*	Complet	
	Merge cand nu avem muchii negative						Neoptim	
Bellman Ford	O(VE)					A* (varianta a BF*)	Complet	
	Detectie Cicluri						Optim cu Euristica	
Floyd-Warshall	O(V^3) - Timp							
	O(V^3) - Spatiu							
	O(V^2) - Spatiu optimizat cu Inchidere Tranzitiva							
Johnson	O(V^2logV + VE)	Grafuri Rare						

Subiectul 3:

Intr-o tara mare producatoare de pucuri de hochei, vrem sa transportam cantitatea maximala de pucuri intre fabrica S si echipa de hochei a statului, T, folosind maxim k camioane. Se cunosc numarul de orase din tara n, numarul de drumuri intre acestea m si numarul maxim de pucuri ce poate fi transportat pe fiecare drum de catre un camion c[i][j], cu i, j = 1..n. Un camion poate transporta oricat de multe pucuri, singura conditie asupra numarului de pucuri fiind data de drumul parcurs de camion.
Se stie: k << n, m

O companie este alcatuita din N soldati. Se considera ca soldatii sunt identificati prin numerele 1, 2, ..., N si se cunosc inaltimile H[i]. La prima inspectie, soldatii sunt asezati in sir in ordinea crescatoare a identificatorului. Capitanul doreste ca din sir sa iasa un numar minim de soldati, iar cei care raman, fara sa-si schimbe locurile, sa poata vedea ambele extremitati ale sirului (din stanga si din dreapta). Un soldat vede o extremitate daca intre el si extremitate nu se afla un soldat cu inaltimea mai mare sau egala cu inaltimea lui. Determinati numarul minim de soldati x care trebuie sa paraseasca sirul pentru a fi indeplinita conditia de mai sus.

Se dau 3 vectori sortati de numere. Se cere sa se aleaga un triplet (cate un numar din fiecare vector) astfel incat sa fie minimizata diferenta maxima dintre oricare 2 numere ale tripletului.

Harta rutiera a unui oras este modelata ca un graf neorientat, presupunandu-se ca se poate circula pe orice strada in ambele sensuri. Primaria vrea sa determine care din aceste strazi vor fi modernizate pentru a forma soseaua de centura. Dupa lungi dezbateri, ajung la urmatoarea decizie pentru alegerea centurii: se aleg strazile care trec prin cat mai multe intersectii din oras si care formeaza un ciclu. Ajutati primaria orasului sa gaseasca strazile ce formeaza soseaua de centura.

Intr-o pădure, toate cărările au aceeaşi lungime, iar Ursul tocmai trezit din hibernare vrea să ajungă de la bârlogul său până la râu cât mai repede întrucât îi este foame şi sete. Însă Vulpea vrea să îi joace o festă şi să îi blocheze Ursului toate căile cele mai scurte care îl duc la râu pentru a îl face să ocolească. Din păcate, pădurea este complicată şi pot exista mai multe drumuri de aceeaşi lungime, iar Vulpea poate alege să blocheze doar o singură intersecţie din pădure. Scopul vostru este să ajutaţi vulpea să decidă dacă există astfel de intersecţii care, dacă ar fi blocate, garantează că ursul va merge o cale mai lungă decât în mod normal.

La intrare primiţi n – numărul de intersecţii din pădure, $a[i][j]$, $1 \le i, j \le n$, matricea de adiacenţă a pădurii, s nodul/intersecţia unde se află bârlogul Ursului şi t nodul/intersecţia unde Ursul preferă să se hrănească la râu. Ieşirea programului o reprezintă acele noduri care satisfac cerinţa din enunţ sau -1 dacă nu există nici un astfel de nod.

Sursa problemei: clasică, apare în mai multe cărţi, forumuri, probleme de studiu, etc.

SUBIECTUL 3 (9p) – 15 min

Se consideră 16 pătrate egale cu laturile colorate în patru culori numerotate de la 1 la 4. Acelaşi pătrat poate avea aceeaşi culoare pe 2 sau mai multe laturi. Să se scrie un program cât mai eficient care primeşte o listă de 16 pătrate şi construieşte o aranjare a lor sub formă de matrice de 4x4 astfel încât două laturi adiacente au aceeaşi culoare.

De exemplu:

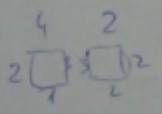
Se dau pătratele de mai jos. 4 4 1 1 înseamnă pătratul 4
1 4

4

4 4 1 1	1 1 3 1	3 2 4 1	4 3 3 2
4 3 1 2	1 3 1 1	1 3 2 2	2 1 1 3
2 2 2 3	2 4 1 3	1 4 4 3	4 4 3 1
4 1 3 2	3 1 1 4	1 1 2 4	2 4 4 4

O soluţie este:

4 2 4 3
2 3 2 1 2
1 2 3 4
2 3 4 1 4
2 1 1 1
4 4 1 3 4
4 2 1 4
2 3 1 1 4
3 1 3 3



Se da un triunghi de numere de forma (numarul de linii este egal cu n):

4
1 7
2 9 6
3 7 1 5

Vrem sa rezolvam urmatorul joc: pornind din varful triunghiului, la fiecare pas trebuie sa cobor un nivel si pot sa aleg sa merg ori in stanga ori in dreapta pe nivelul imediat inferior. In functie de alegerea facuta, scorul creste cu modulul diferentei intre numarul curent si numarul ales de pe nivelul inferior. Scopul acestui joc este maximizarea scorului cu care se ajunge pe ultimul nivel. Construiti un algoritm pentru rezolvarea optima a jocului.
Exemplu de drum ales si de scor: $4 \rightarrow 1 \rightarrow 9 \rightarrow 1$ are scorul $(4-1) + (9-1) + (9-1) = 3 + 8 + 8 = 19$

SUBIECTUL 4 (7p) – 15 min

Sa se un arbore cu n varfuri si fara costuri si se doreste determinarea diametrului acestui arbore. Diametrul unui graf (deci, si al unui arbore) este reprezentat de catre cea mai lunga cale cea mai scurta intre oricare 2 varfuri ale grafului. Se schimba rezolvarea daca avem de a face cu un graf si nu cu un arbore?

SUBIECTUL 5 (7p)

Dandu-se doua stringuri A si B, calculati distanta minima de editare dintre A si B. Distanta minima de editare dintre A si B reprezinta numarul minim de operatii necesare transformarii stringului A in stringul B. Operatiile care se pot realiza sunt de **inserare** a unei litere, de **stergere** a unei litere sau de **inlocuire** a unei litere cu o alta.

Intr-o pădure, toate cărările au aceeași lungime, iar Ursul tocmai trezit din hibernare vrea să ajungă de la bârlogul său până la râu cât mai repede întrucât îi este foame și sete. Însă Vulpea vrea să îi joace o festă și să îi blocheze Ursului toate căile cele mai scurte care îl duc la râu pentru a îl face să ocolească. Din păcate, pădurea este complicată și pot exista mai multe drumuri de aceeași lungime, iar Vulpea poate alege să blocheze doar o singură intersecție din pădure. Scopul vostru este să ajutați vulpea să decidă dacă există astfel de intersecții care, dacă ar fi blocate, garantează că ursul va merge o cale mai lungă decât în mod normal.

La intrare primiți n – numărul de intersecții din pădure, $a[i][j]$, $1 \le i, j \le n$, matricea de adiacență a pădurii, s nodul/intersecția unde se află bârlogul Ursului și t nodul/intersecția unde Ursul preferă să se hrănească la râu. Ieșirea programului o reprezintă acele noduri care satifac cerința din enunț sau -1 dacă nu există nici un astfel de nod.

Sursa problemei: clasică, apare în mai multe cărți, forumuri, probleme de studiu, etc.

Broscuța Oac trebuie să traverseze un râu care are distanța de la un mal la altul de fix n metri. Pentru a face acest lucru, Oac trebuie să sară de pe o piatră pe alta, însă are o problemă cu adaptabilitatea: dacă săritura curentă este de x metri, săritura următoare poate fi între $x-1$ și $x+1$ metri. Știind că prima săritură a broscuței este întotdeauna de exact un metru, precum și că Oac a descoperit un șir liniar de pietre discuse la un număr întreg de metri de malul de pe care se află, determinați dacă poate face un șir de sărituri care să o ducă pe malul opus. Șirul liniar de pietre este dat sub forma unui vector *pietre[1..n-1]* cu următoarea semnificație: *pietre[i]=1* dacă există o piatră la i metri de malul pe care se află inițial Oac, și *pietre[i]=0* altfel. Dacă există un șir corect de sărituri pentru traversarea râului, afișați și pietrele pe care le-a folosit Oac pentru a ajunge de la un mal la altul.

Sursa problemei: Aziz, Adnan, and Amit Prakash. *Algorithms For Interviews*. CreateSpace, 2010.

Ionel a invatat urmatorul joc de la prietenii sai. Dandu-se un pachet de **N** carti, cu cate un numar (pozitiv sau negativ) inscris pe fiecare carte (**carte[i], 1≤i≤N**), cei doi jucatori vor lua alternativ 1, 2 sau 3 carti din varful pachetului. Numerele de pe fiecare carte extrasa de catre un jucator sunt adunate la scorul acestora. Stiind ca initial ambii jucatori au scorul 0 si ca Ionel va incepe jocul de fiecare data, determinati care este scorul maxim pe care Ionel il poate atinge. Atat Ionel, cat si partenerul sau, vor juca optim pentru maximizarea propriului scor.

Sursa problemei: HackerRank, infoarena, este o problema destul de clasica.

Algoritm Tarjan (I)

tarjan

Anterior

- Articulații (G)
  - V = noduri(G) // inițializări
  - Timp = 0;
  - **Pentru fiecare** (u ∈ V)
    - c(u) = alb;
    - d(u) = 0;
    - p(u) = null;
    - low(u) = 0;
    - subarb(u) = 0; // reține numărul de subarbori dominați de u
    - art(u) = 0; // reține punctele de articulație
  - **Pentru fiecare** (u∈V)
    - **Dacă** c(u) e alb
      - Exploreaza(u);
      - **Dacă** (subarb(u) > 1) // cazul în care u este rădăcina în arborele
        - art(u) = 1 // DFS și are mai mulți subarbori → cazul // 1 al teoremei

Algoritm Tarjan (II)

tarjan

Anterior

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v ∈ succs(u)
    - **Dacă** (c(v) e alb)
      - p(v) = u; subarb(u)++; // actualizare nr subarbori // dominați de u
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (p(u) ≠ null && low(v) ≥ d(u)) art(u) = 1; // cazul 2 al teoremei
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low

PA

Algoritmul lui Tarjan adaptat pentru determinarea CTC

Căutare în doc
tarjan
Anterior

- index = 0 // nivelul pe care este nodul în arborele DFS
- S = empty // se folosește o stivă care se inițializează cu Ø
- **Pentru fiecare** v din V
  - **Dacă** (v.index e nedefinit) **atunci** // se pornește DFS din fiecare nod pe care
    - Tarjan(v) // nu l-am vizitat încă

- Tarjan(v)
  - v.index = index // se setează nivelul nodului v
  - v.lowlink = index // reține strămoșul nodului v
  - index = index + 1 // incrementez nivelul
  - S.push(v) // introduc v în stivă
  - **Pentru fiecare** (v, v') din E // se prelucrează succesorii lui v
    - **Dacă** (v'.index e nedefinit **sau** v' e în S) **atunci** // CTC deja identificate sunt ignorate
      - **Dacă** (v'.index e nedefinit) **atunci** Tarjan(v') // dacă nu a fost vizitat v' intru în recursivitate
      - v.lowlink = min(v.lowlink, v'.lowlink) //actualizez strămoșul
  - **Dacă** (v.lowlink == v.index) **atunci** // printez CTC începând de la coadă spre rădăcină
    - print "CTC:"
    - **Repetă**
      - v' = S.pop // extrag nodul din stiva și îl printez
      - print v'
    - **Până când** (v' == v) // până când extrag rădăcina

Algoritm punți (I)

Căutare în docu
tarjan
Anterior

- Punți(G)
  - V = noduri(G) // inițializări
  - Timp = 0;
  - **Pentru fiecare** nod u (u ∈ V)
    - c(u) = alb;
    - d(u) = 0;
    - p(u) = null;
    - low(u) = 0;
    - punte(u) =0; // înlocuiește: subarb(u) = 0; art(u) = 0;
  - **Pentru fiecare** nod u (u ∈ V)
    - **Dacă** c(u) e alb
      - Explorează(u)

PA

Algoritm punți (II)

- Explorează(u)
  - d(u) = low(u) = timp++; // inițializări
  - c(u) = gri;
  - **Pentru fiecare** nod v (v ∈ succs(u))
    - **Dacă** c(v) e alb
      - p(v) = u; // se elimină: subarb(u)++;
      - Explorează(v);
      - low(u) = min{low(u), low(v)} // actualizare low
      - **Dacă** (low(v) > d(u)) punte(v) = 1;

      // în loc de: **Dacă**(p(u) != null && low(v) ≥ d(u))
    - **Altfel**
      - **Dacă** (p(u) ≠ v) low(u) = min{low(u), d(v)} // actualizare low